GOOGLE INTERVIEW EXPERIENCE

Complete Interview Experience for Google SDE Role

RESOURCE	DETAIL
Type	Interview Experience / Preparation Guide
Category	Campus Placement • Software Engineering
Format	Round-by-round interview walkthrough
Coverage	OA • Technical Screen • DSA • System Design • Behavioral
Language	English

Important note: This is a structured, representative Google-style interview experience based on the supplied outline. It should not be treated as a verified account of a specific candidate's actual interview.

Reality check: Interview loops vary by role, level, location, team and hiring cycle. Use the question types and preparation framework as guidance rather than as a guaranteed question list.

CANDIDATE PROFILE

Name	Priya Patel
Target Role	Software Development Engineer
College	IIT Delhi
Batch	2024
Preparation Focus	Algorithms, Data Structures, DP, System Design, Behavioral

Preparation philosophy: Build strong fundamentals first, then practice hard problems under time pressure. For every solution, explain the idea, prove why it works, analyze complexity and test edge cases.

INTERVIEW ROUND OVERVIEW

Round	Duration	Main focus	Difficulty
1. Online Assessment	90 min	2 coding + 2 DP + 10 algorithm MCQs	Hard
2. Technical Phone Screen	45 min	Median of two sorted arrays + autocomplete design	Medium–Hard
3. Onsite — DSA	60 min	K-th smallest in matrix + Word Search II + graph traversal	Hard
4. Onsite — System Design	60 min	Google Drive + YouTube design discussion	Hard
5. Onsite — Behavioral	45 min	Leadership, conflict, project management	Medium

What interviewers generally look for: algorithmic reasoning, correctness, efficiency, communication, design trade-offs, debugging ability and collaboration.

ROUND 1 — ONLINE ASSESSMENT

Duration: 90 minutes

- **2 Hard Coding Problems:** Expect problems that test algorithm selection, implementation and complexity.
- **2 Dynamic Programming Problems:** Practice identifying state, transition, base cases and optimization.
- **10 Algorithm MCQs:** Revise complexity, sorting, searching, graphs, trees and fundamental data structures.
- **Time management:** Quickly identify problems with a clear path to solution and avoid getting trapped on one problem.

Preparation tip: Practice mixed timed sets instead of solving only one topic at a time. The assessment tests how quickly you recognize a pattern under pressure.

TECHNICAL SCREEN — MEDIAN OF TWO SORTED ARRAYS

Core challenge: Find the median of two sorted arrays without simply merging them.

- **Naive approach:** Merge both arrays and find the median — O(m+n).
- **Optimized approach:** Binary search a partition in the smaller array so that the left side contains half the total elements and every left value is ≤ every right value.
- **Key insight:** Once the correct partition is found, the median is determined by the maximum value on the left and minimum value on the right.
- **Edge cases:** Empty one-side array, even/odd total length, duplicate values and very different array sizes.

Interview follow-up: Why binary search the smaller array? What are the exact partition boundaries? How do you handle even total length?

TECHNICAL SCREEN — SEARCH AUTOCOMPLETE SYSTEM

Basic architecture: Client → API service → prefix-search component → storage/cache → ranked suggestions.

Component	Purpose
Trie / Prefix index	Efficiently finds terms matching a typed prefix
Ranking layer	Orders suggestions using popularity, relevance or personalization
Cache	Reduces repeated lookup latency for common prefixes
Storage	Persists the dictionary, metadata and ranking information
API service	Receives prefix requests and returns suggestions

Design follow-ups: How do you handle millions of queries? How do you update rankings? How do you cache hot prefixes? How do you filter abusive or inappropriate suggestions?

ROUND 3 — ONSITE DSA

Problem	Core technique	What to explain
K-th Smallest in Matrix	Heap or binary search on value range	Sorted rows/columns, complexity trade-off
Word Search II	Trie + DFS/backtracking	Prefix pruning and visited-cell handling
Graph Traversal	BFS/DFS	Visited state, components, shortest path when applic

Word Search II insight: Running a separate DFS for every word can repeat large amounts of work. A Trie lets the search stop immediately when the current path is not a prefix of any target word.

ROUND 4 — ONSITE SYSTEM DESIGN

SYSTEM DESIGN QUESTION 1 — GOOGLE DRIVE

- **Requirements:** upload/download files, folders, sharing, permissions, version history and search.
- **Core services:** API gateway, metadata service, object storage, database, authorization service, search index and notification/event pipeline.
- **Storage:** Store file metadata separately from large file blobs. Object storage is appropriate for large immutable chunks.
- **Scalability:** Chunk large files, support resumable uploads, cache metadata and horizontally scale stateless services.
- **Reliability:** Replication, checksums, retry logic and durable metadata are important.

SYSTEM DESIGN QUESTION 2 — YOUTUBE

- **Core flow:** upload → processing/transcoding → storage → CDN → playback.
- **Transcoding:** Generate multiple resolutions/bitrates so clients can use adaptive streaming.
- **CDN:** Cache popular video segments close to viewers to reduce latency and origin load.
- **Metadata:** Store title, creator, privacy, thumbnails, categories and processing state.
- **Scale considerations:** Huge storage volume, high read traffic, asynchronous processing, recommendation/search systems and abuse controls.

Design rule: Start with requirements and scale assumptions before drawing architecture. Then discuss data flow, bottlenecks, failure modes and trade-offs.

ROUND 5 — ONSITE BEHAVIORAL

Area	Preparation
Leadership	Prepare examples of taking ownership, influencing others and delivering a difficult g
Conflict Resolution	Show how you listened, clarified the disagreement and reached a constructive outc
Project Management	Explain planning, prioritization, risks, deadlines and communication.
Failure / Mistake	Own the mistake, explain the root cause and show the concrete improvement afterv
Ambiguity	Explain how you gathered information, made assumptions and moved forward resp

Behavioral answer framework: Situation → Task → Action → Result → Reflection. The Action section should make your individual contribution clear.

GOOGLE-STYLE DSA PREPARATION CHECKLIST

- ✓ Arrays, strings and hashing
- ✓ Two pointers and sliding window
- ✓ Binary search and search-space binary search
- ✓ Linked lists, stacks and queues
- ✓ Trees, BSTs and heaps
- ✓ Graphs: BFS, DFS, shortest paths and topological sort
- ✓ Recursion and backtracking
- ✓ Dynamic programming: 1D, 2D, subsequence and knapsack patterns
- ✓ Tries and prefix-based problems
- ✓ Intervals, greedy algorithms and monotonic stacks
- ✓ Time and space complexity
- ✓ Writing clean, testable code without overengineering

Mock interview habit: Practice explaining the solution while coding. A technically correct answer that cannot be communicated clearly is harder for an interviewer to evaluate.

6-MONTH PREPARATION ROADMAP

Month	Primary focus
1	DSA fundamentals: arrays, strings, hashing, linked lists, stacks and queues
2	Trees, heaps, graphs, BFS/DFS and binary search
3	Dynamic programming, recursion, backtracking, tries and greedy
4	Hard mixed DSA sets + timed coding practice + complexity review
5	System design basics, APIs, databases, caching, scaling and mock interviews
6	Full interview simulations, behavioral stories, weak-topic revision and company-specific research

Final preparation checklist: DSA fundamentals ✓ • hard problems ✓ • DP ✓ • system design ✓ • mock interviews ✓ • behavioral stories ✓ • clear communication ✓ • edge-case testing ✓ • complexity analysis ✓